

Lab 2

Zagadnienia do opanowania:

- UE5 podstawowe klasy
 - UE5 enhanced input system
-

Zadania:

1. Utworzyć nowy projekt: szablon Blank (Blueprint, Desktop, Maximum, bez Starter Content i bez RealTime). Dodać nową mapę typu Basic.
2. Tworzenie podstawowych klas:
 - 2.1. Character – Lab1Character
 - 2.2. GameMode: Lab1GM. Ustawiamy Default Pawn Class nasz utworzony przed chwilą Lab1Character.
 - 2.3. Ustawiamy jako domyślny tryb gry (GM) utworzoną klasę Lab1GM. WorldSettings Game Mode Override.
 - 2.4. Enhanced Input System – sterowanie postacią:
 - 2.4.1. Tworzymy akcję (Input->InputAction) IA_MoveForward. Ustawiamy ValueType na Axis1D (float)
 - 2.4.2. Tworzymy Mapping Context (Input->Input Mapping Context). Dodajemy Mappings i wybieramy utworzoną akcję. Następnie mapujemy klawisze (np. W S). Do S dodajemy Modifiers Negate.
 - 2.4.3. Przechodzimy do Lab1Character i w zdarzeniu BeginPlay podpinamy nasz Enhanced Input Local Player Subsystem. Pobieramy Controller (funkcja GetController) rzutujemy na PlayerController i następnie z Player Controller wybieramy funkcję GetEnhancedInputLocalPlayerSubsystem. Sprawdzamy jej poprawność (IsValid) a następnie dodajemy nasz kontekst (AddMappingContext). Sprawdzamy działanie klawiszy i akcję MoveForward. Funkcje IA_MoveForward, PrintString i Append dla Stringów. Zrobić samemu drugą akcję MoveRight.
 - 2.4.4. Akcję poruszania można zrobić od razu całą (tj. zintegrować MoveForward i MoveRight). Utworzyć nową akcję IA_Move. Ustawić typ na Axis2D (Vector2D) i w modifiers dodać SwizzleInputAxisValue. Następnie w Mapings (w IMC) dodać do akcji IA_Move cztery klawisze (np. strzałki). Do akcji Up/Down dodajemy modifier SwizzleInputAxisValue. Dodatkowo do Down i Left dodajemy Negate. Obsłużyć działanie w Lab1Character. Uwaga: Action Value dla IA_Move można rozbić na dwie zmienne PPM->Split Struct Pin).
 - 2.4.5. Poruszanie postacią. Pobieramy funkcje: GetControlRotation i GetForwardVector. Następnie rozdzielamy pin wyjściowy (PPM na Return Value albo In Rot i Split Struct Pin) i podłączamy Z(Yaw) value. Następnie return value trafia do AddMovementInput. ScaleValue bierzemy z AxisValue zdarzenia IA.
 - 2.4.6. Modyfikujemy wygląd postaci. Dodajemy do komponentów: 2x static mesh (cube i cone) i Camera. Skalujemy, obracamy i ustawiamy tak, żeby uzyskać żądany efekt.
 - 2.4.7. Dodajemy akcję IA_Look: Axis2D(Vector2D). W IMC podpinamy od akcji Mouse XY 2D-Axis. Obsługujemy zdarzenie IA_Look w Lab1Character dodając AddControllerYawInput (Value X) i AddControllerPitchInput (Value Y). Żeby działało poruszanie kamerą góra/dół trzeba ustawić dla obiektu kamery opcję UsePawnControlRotation na true.

3. Tworzenie sceny z geometrii

- 3.1. W zakładce Place actors przechodzimy do panelu geometry (trzeci od końca) i dodajemy Box. W jego właściwościach można ustawić rozmiar. Następnie w Combo box zaraz pod głównym menu wybieramy Brush Editing (SHIFT+7) i możemy naszą geometrię edytować.
- 3.2. Sprawdzić narzędzia: Pen (uwaga na widok – tutaj najlepiej przełączyć się na widok z góry), Edit, Extrude.
- 3.3. Sprawdzić działanie Geomtry gdy jego Brush Type będzie typu Subtractive, a nie Additive (domyślnie) – zmieniamy w panelu Details geometrii.

4. Mierzenie czasu zakończenia gry

Umieszczamy na mapie aktora typu TriggerBox. Mając go zaznaczonego otwieramy level blueprint (trzeci combo box – schemat?) i opcja Open Level Blueprint. Następnie klikamy PPM i na samej górze dostępnych funkcji mamy Add Event for Trigger... rozwijamy, Collision->AddOn ActorBeginOverlap. W reakcji na to zdarzenie wyświetlamy czas, który upłynął od rozpoczęcia gry (GetRealTimeSeconds), następnie wyłączamy sterowanie (SetInputModeUIOnly z podpiętym kontrolerem z funkcji GetPlayerController). Dodajemy jakieś opóźnienie (Delay) i wychodzimy z gry (QuitGame).

5. Utworzenie klasy przeciwnika, który będzie się poruszał i w momencie zderzenia naszego gracza z przeciwnikiem tracimy znacząco na prędkości poruszania się, która będzie nam stopniowo wracała.

5.1. Wykrywanie zderzenia i obniżanie prędkości

- 5.1.1. Tworzymy nowego aktora dziedziczącego po klasie Actor. Nazywamy go Enemy. Ustawiamy mu jakiś wygląd (dodajemy do komponentów StaticMesh i wybieramy w Details np. Cylinder). Można również pobawić się z teksturą. Ustawiamy przeciwnika na scenie żebyśmy mogli przeprowadzić testy.
- 5.1.2. Otwieramy Lab1Character i dodajemy trzy nowe zmienne: MaxSpeed(float) i LimitedSpeed(float). Ich znaczenie jest następujące. MaxSpeed to maksymalna prędkość naszego aktora (domyślnie początkowa, czyli 600), a druga LimitedSpeed to prędkość aktora po zderzeniu z przeciwnikiem. Inicjujemy wartość MaxSpeed dodając kod (SetMaxSpeed) na CharacterMovement->GetMaxWalkSpeed.
- 5.1.3. Wykrywamy zderzenie (Hit) i z pinu Other próbujemy rzutować aktora z którym się zderzyliśmy na Enemy. Jeśli się to rzutowanie uda, to ustawiamy CharacterMovement->SetMaxWalkSpeed na LimitedSpeed, a następnie tworzymy nowy Timeline (Add Timeline). Klikamy dwukrotnie na tym komponentcie i tworzymy nowy Track o nazwie SpeedRecovery. Ustawiamy długość na (3? – po ilu sekundach po zderzeniu chcemy odzyskać pełną prędkość). Następnie dodajemy dwa punkty Shift-LPM i edytujemy punkt Time: 0 Value 1. Drugi punkt powinien mieć wartości Time: 3, Value 1 (to 3 przy założeniu długości tego track'a). Podłączmy sterowanie pod PlayFromStart. Pin wyjściowy Update podpinamy do SetMaxWalkSpeed komponentu CharacterMovement. Wartość bierzemy z funkcji Lerp (float) podpinając pod A MaxSpeed, pod B LimitedSpeed, a pod Alpha SpeedRecovery. Możemy jeszcze kontrolnie podpiąć pod pin Finished ustawienie SetMaxWalkSpeed na MaxSpeed.
- 5.1.4. Zastanowić się (można testowo to wykonać) co trzeba by było zrobić, żeby kolejne ograniczenie prędkości można było otrzymać dopiero po zakończeniu poprzedniego.

5.2. Poruszanie przeciwnikiem.

- 5.2.1. Dodajemy do blueprintu przeciwnika dwie zmienne typu Vector o nazwach

StartPosition i EndPosition. EndPosition robimy Publiczne i zaznaczmy mu dodatkowo Show3DWidget. Ustawiamy StartPosition na początkową pozycję aktora (funkcja GetActorLocation). Następnie tworzymy nowy Timeline, dodajemy nowy FloatTrack i dwa punkty: (0,0) i (5,1) – tutaj zakładamy, że przeciwnik będzie pokonywał drogę z punktu StartPoint do EndPoint w 5 sekund.

- 5.2.2. Update z timeline podpinamy do SetActorLocation. Z NewLocation wyciągamy funkcję Lerp (Vector). Jako A ustawiamy StartPosition, a jako B StartPosition+EndPosition (EndPosition jest względem pozycji naszego aktora!). Następnie wyciągamy z pinu Direction wpisujemy Switch on ETimelineDirection. Do tego węzła podpinamy Finished Timeline'u. Pin Forward ze switcha podpinamy do Reverse from End, a pin Backward do Play from Start.